

AWS IAM (Identity and Access Management)

What is IAM?

- **Definition:** A web service that helps you **securely control access** to AWS resources.
- **Authentication (AuthN):** Verifying **who** is making the request (Sign-in credentials, Access keys).
- **Authorization (AuthZ):** Determining **what** actions they are allowed to perform.
- **Scope: Global service** (identities and policies apply across all AWS regions automatically).
- **Cost:** Offered at **no additional charge** (you only pay for resources created by IAM identities).
- **Real-World Use Case:** A financial application requires developers to deploy code but blocks them from viewing production transaction databases.

Core Components

The Building Blocks of IAM

- **IAM User:** A persistent identity representing a **single person or application** needing direct interaction with AWS.
- **Credentials:** Can hold up to two active access keys (CLI/API) and one password (Management Console).
- **IAM Group:** A **collection of IAM users** used to specify permissions for multiple users simultaneously.
- **Group Constraints:** Groups cannot be nested; they contain only users, not other groups.
- **Real-World Use Case:** An organization creates a SysAdmin group with full infrastructure access and a QA-Engineers group with read-only access. New hires are simply added to the appropriate group.

Dynamic and Temporary Permissions

- **IAM Role:** An identity with **permission policies** that determine what the identity can do in AWS, featuring **no permanent credentials**.
- **Temporary Credentials:** Provides short-lived security tokens via the AWS Security Token Service (STS).
- **IAM Policy:** An object that, when associated with an identity or resource, **defines their permissions**.
- **Real-World Use Case:** A mobile application authenticates users via Google Sign-In, and AWS IAM gives them temporary, restricted access to their specific user folder in Amazon S3.

Policy Document Format and Structure

Anatomy of a JSON Policy

Format: Written strictly in **JSON (JavaScript Object Notation)**.

- **Version:** Element defining the policy language version (always use "2012-10-17").
- **Statement:** The main policy container. Can include a single statement or an array of individual statements.
- **Structural Elements:**
 - Sid:** (Optional) Statement ID to describe the purpose.
 - Effect:** Specifies whether the statement results in an Allow or Deny.
 - Principal:** Indicates **who** is allowed or denied access (Required only in resource-based policies).
 - Action:** Specific API operations to allow or deny (e.g., s3:GetObject).
 - Resource:** The specific AWS resources the actions apply to, using **Amazon Resource Names (ARNs)**.
 - Condition:** (Optional) Circumstances under which the policy is valid (e.g., IP address ranges, Time of day).

```
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Sid": "AllowS3ReadWithMFA",
    "Effect": "Allow",
    "Action": ["s3:GetObject",
               "s3:ListBucket"],
    "Resource": ["arn:aws:s3:::company-
                  data-bucket/*"],
    "Condition": {      "Bool":
                        {"aws:MultiFactorAuthPresent":
                         "true"}    }
    }
  ]
}
```

Resource Control Policies (RCPs) vs Service Control Policies (SCPs)

Data-Centric Perimeter Guardrails

- **Definition:** AWS Organizations Service Control Policies (SCPs) set maximum permission boundaries across accounts, while **Resource Control Policies (SCPs/RCPs)** centrally restrict data access on resources.
- **Mechanics:** They do not grant permissions. Instead, they act as a **filter** that limits what identities within an AWS Organization can do.
- **Hierarchy:** Applied at the Organization Root, Organizational Units (OUs), or specific Member Accounts.
- **Real-World Use Case:** A multinational corporation enforces a policy at the Organization root level that explicitly prevents any developer in any account from deleting AWS CloudTrail logs.

Implicit Deny vs. Explicit Deny

Understanding the Default State

- **Default Security State:** All requests are **denied by default**. This is known as an **Implicit Deny**.
- **Implicit Deny:** If no policy explicitly grants permission for an action, the request is automatically blocked. It can be overridden by an Allow.
- **Explicit Deny:** A policy statement containing "Effect": "Deny".
- **The Golden Rule:** An **Explicit Deny always trumps an Allow**, regardless of where it is evaluated or which policy contains it.
- **Real-World Use Case:** A user belongs to a group that has an Allow policy for all EC2 actions. However, a separate policy applied directly to the user contains an Explicit Deny for ec2:TerminateInstances. The user cannot terminate instances.

Policy Evaluation Logic

How AWS Decides to Approve or Block a Request

- **Step 1: Initialization** – AWS compiles all policies applicable to the request context (Identity-based, Resource-based, SCPs, Boundaries).
- **Step 2: Check for Explicit Deny** – If any policy evaluated contains a matching Deny statement, the evaluation ends immediately with a final decision of **Deny**.
- **Step 3: Check for Organization Policies** – AWS checks SCPs/RCPs. If not allowed here, the request results in an implicit deny.
- **Step 4: Check for Permissions Boundaries** – Validates if the action fits within the user's defined boundary limits.
- **Step 5: Check for Explicit Allow** – Looks for an Allow statement in Identity-based or Resource-based policies. If found, the final decision is **Allow**.
- **Step 6: Final Fallback** – If no explicit Allow is discovered through the evaluation layers, the request lands back on the default **Implicit Deny**.

Security Features

- MFA (Multi-Factor Authentication)
- Access Keys
- Password Policy enforcement